

Practical 2

AIM: Implementation and Time analysis of linear and binary search algorithm.

1 ALGORITHM:

Linear search:- (Algorithm)

Start from the first element
Compare each element with the key
If found $\rightarrow$ return position
If not $\rightarrow$ move to next.
End after last element (not found).

Binary search

work on a sorted array
Set low and high pointers.
Find $mid = (low + high) / 2$.
If $key = mid \rightarrow$ return position.
If $key < mid \rightarrow$ search left half.
If $key > mid \rightarrow$ search right half.
Repeat until low $>$ high (not found).

2 PROGRAM:

```
#include <iostream>
#include <vector>
using namespace std;
```

```
// Linear Search Function
```

```
int linearSearch(vector<int> arr, int key) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
```

```
// Binary Search Function
```

```
int binarySearch(vector<int> arr, int key) {
```

```
int low = 0;
int high = arr.size() - 1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] == key)
        return mid;
    else if (arr[mid] < key)
        low = mid + 1;
    else
        high = mid - 1;
}
return -1;
}

int main() {
    int n = 100; // smaller size for easy testing
    vector<int> arr(n);
    // Creating a sorted array
    for (int i = 0; i < n; i++)
        arr[i] = i + 1;

    int key;
    cout << "Enter element to search: ";
    cin >> key;
    int index;
    // Linear Search
    index = linearSearch(arr, key);
    cout << "\nLinear Search:\n";
    if (index != -1)
        cout << "Element found at index " << index << endl;
    else
        cout << "Element not found\n";
    // Binary Search
    index = binarySearch(arr, key);
    cout << "\nBinary Search:\n";
    if (index != -1)
        cout << "Element found at index " << index << endl;
    else
        cout << "Element not found\n";
    return 0;
}
```

3 OUTPUT:

```
C:\WINDOWS\system32\cmd. x + v
Enter element to search: 2
Linear Search:
Element found at index 1
Binary Search:
Element found at index 1
C:\Users\YASWANTH\DAA>
```

4 TIME COMPLEXITY:

Time complexity:- time to search in array of size n.
 $T(n) = T(n-1) + O(1)$

sum

$$T(n) = T(n-1) + 1 = T(n-2) + 1 + 1 = \dots = T(1) + (n-1)$$

Best case: $T(1) = O(1)$.

final:- $T(n) = O(n)$.

Best case: $O(1)$

Worst case: $O(n)$

Average case: $O(n)$

Time complexity

Let $T(n)$ = time to search in array of size n.
 for checking middle element constant $O(1)$.
 Search half the array $\rightarrow n/2$.

$$T(n) = T\left(\frac{n}{2}\right) + 1 = T\left(\frac{n}{4}\right) + 1 + 1 = \dots = T(1) + \log_2 n$$

Best case: $T(1) = O(1)$.

$$T(n) = O(\log n)$$

Best Case = $O(1)$

Average case: $O(\log n)$

Worst case: $O(\log n)$

5 Conclusion: LINEAR SEARCH HAS A TIME COMPLEXITY OF $O(N)$, WHILE BINARY SEARCH RUNS IN $O(\log N)$ ON SORTED ARRAYS.